On-Device Reusable Expression Runtime with paceval

A C++ feasibility study of approved mathematical models, reusable execution graphs and numerical validation

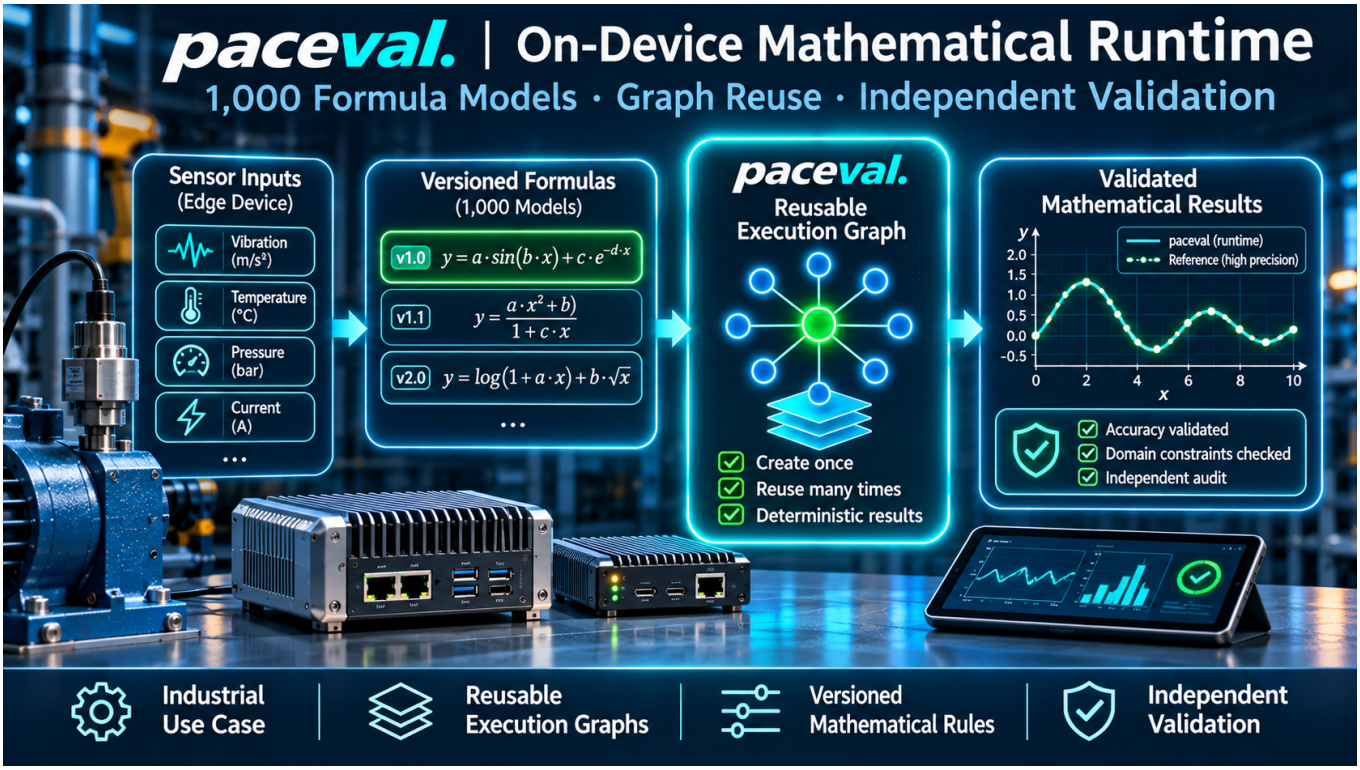


Figure 1. Application illustration. The pictured systems are conceptual; the executed study uses synthetic inputs.

Executive finding. Using the official paceval C++ interface and the original x86-64 Linux runtime, the demonstrator successfully created 1,000 distinct formula-version graphs, checked 32,000 numerical outputs, verified a controlled update and performed a separate pointwise TINC audit. The independent 100-digit audit passed every regular calculation under its stated tolerance.

Research dimension	Verified scope
Formula catalog	1,000 approved expressions from five mathematical structures
Evaluation and audit	32,000 values, native C++ plus independent mpmath oracle
Model maintenance	16 sample checks of an isolated version update
Platform	x86-64 Linux; ARM device execution not tested

1. Motivation and execution architecture

Industrial edge applications may need to evaluate repeatedly used mathematical rules for sensor transformation, engineering correction, scoring and configurable numerical logic. Separating the definition of a mathematical model from its repeated execution can help keep the host application stable when approved formula versions change. The present study tests that separation, not a deployed industrial sensor or a certified real-time runtime.

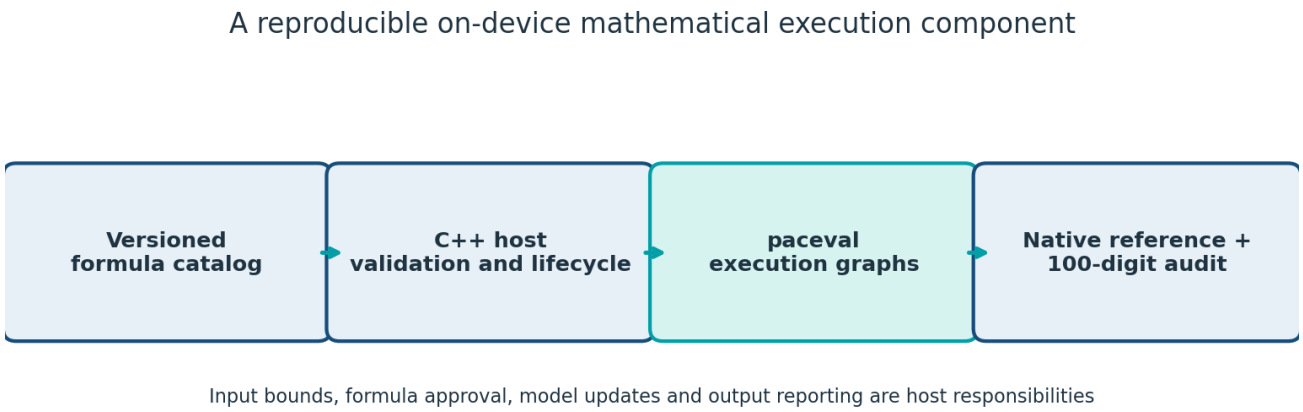


Figure 2. The application host validates inputs and versions, while paceval evaluates prepared mathematical graphs.

The host program fixes the input contract, formula catalog, version metadata and output format; it owns `paceval_CreateComputation` handles and calls `paceval_DeleteComputation` before library shutdown. It calls the official `paceval_dGetComputationResultExt` batch function for repeated values. Its return value is interpreted correctly: the boolean reports the presence of an API error; per-item error codes are also checked.

The project is intentionally in-process. It does not implement an ONNX inference engine, remote REST service, ARM emulator, RTOS, device driver or production device watchdog. The original binary paceval runtime must be supplied independently for the target CPU architecture.

Research boundary: 1,000 distinct approved formulas are tested, but they belong to only five analytic families. This verifies catalog and handle reuse, not unrestricted mathematical complexity or scientific model accuracy.

2. Mathematical workload and inputs

Five bounded mathematical families are parameterized by variables (x, a, b, c, d). The reference starting point from WP-008 is the damped sinusoid plus exponential term. Four further families exercise cosine, rational, polynomial, hyperbolic tangent, logarithm and square root, under strictly positive denominators and function arguments.

Family	Expression (each also includes a version-specific constant offset)
0 - damped sinusoid	a*sin(b*x) + c*exp(-(d*x))
1 - rational/cosine	a*cos(b*x) + c/(1+d*x*x)
2 - polynomial/decay	a*x*x + b*x + c*exp(-(d*x))
3 - tanh/sine	a*tanh(b*x) + c*sin(d*x)
4 - logarithm/root	a*ln(1+b*x*x) + c*sqrt(1+d*x*x)

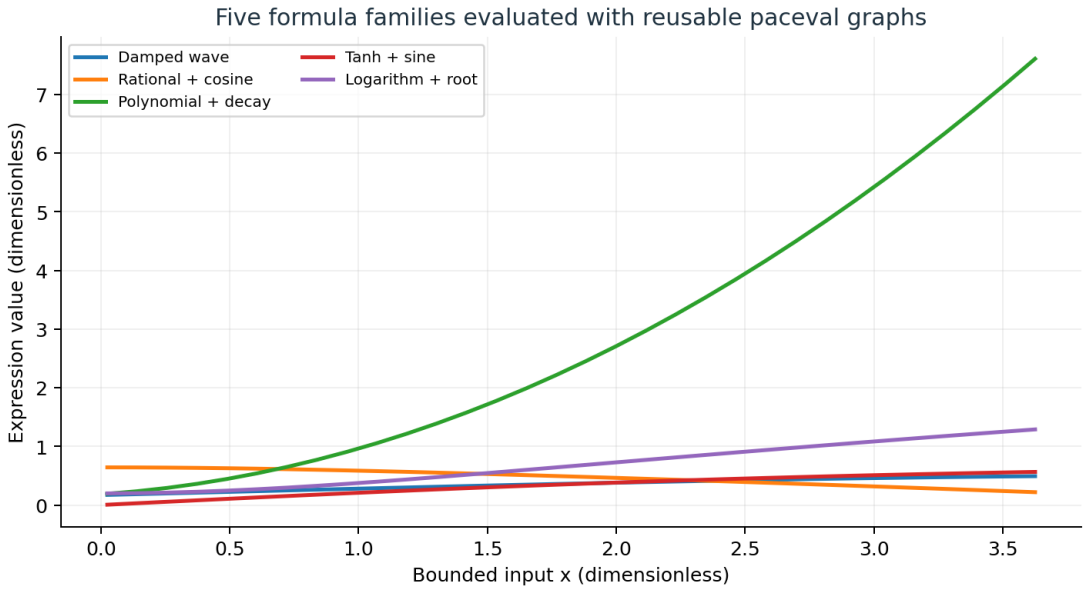


Figure 3. Example results on the synthetic bounded domain, from the published evaluation CSV.

The generated inputs vary x and the four coefficients deterministically. x starts at 0.025 and moves in positive increments; coefficients remain bounded and positive. All quantities in this synthetic benchmark are dimensionless. The mathematical score is not claimed to be a physical measurement.

3. Approved catalog and graph lifecycle

The published run creates 1,000 separate paceval computation objects, distributing 200 formula versions across each of five base structures. Each version has its own nine-decimal constant embedded in the expression text. Handles are stored using C++ ownership semantics and released automatically. A fixed 32-sample input sweep is evaluated for each graph, resulting in 32,000 checked values.

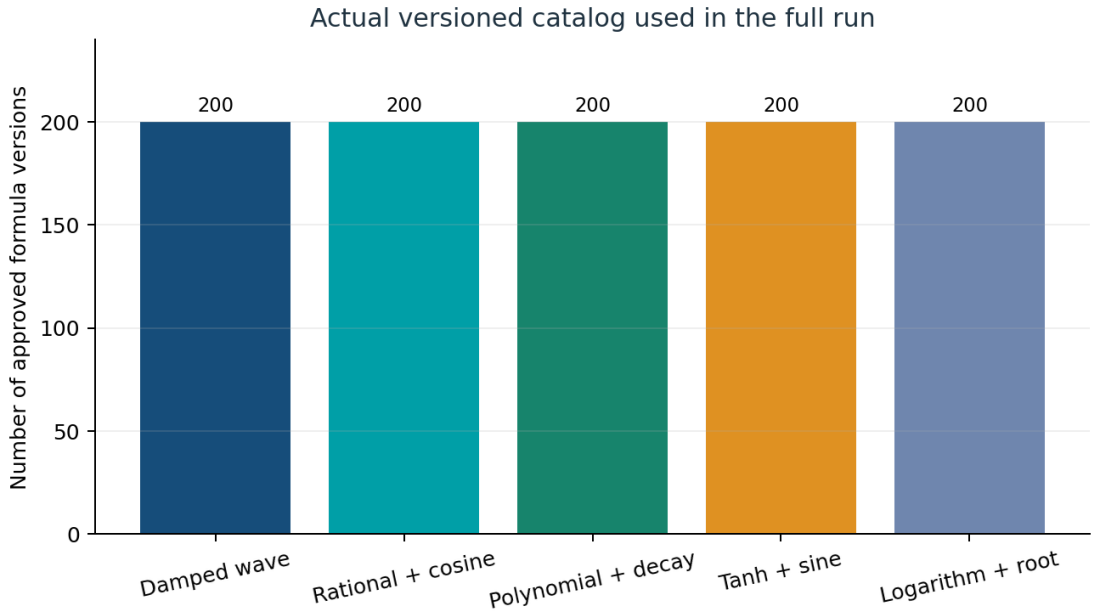


Figure 4. Actual distribution of the prepared 1,000-formula catalog.

For a formula update, the host first creates the replacement computation object. Only when that graph succeeds does the single-threaded host replace the selected stored handle. The test updates model 17, with an additive change of +0.125, and verifies 16 corresponding old/new output pairs. Other handles are not re-created in that operation.

The C++ demonstration rejects unknown command-line arguments, invalid negative model counts, values outside configured batch limits and nonfinite evaluation inputs before passing them to the engine. Limits used by this research CLI are operational policies, not universal restrictions imposed by paceval.

4. Numerical verification

Every published regular calculation was evaluated by paceval and an independent native C++ expression. Separately, a Python implementation using mpmath with 100 decimal digits recomputed all 32,000 output values from the published input CSV. The evaluator uses the exact literal version offset and the floating-point input values, ensuring the comparison is well specified.

Numerical criterion	Observed result
Accepted values	32,000 of 32,000 within 2e-11 × (1 + |reference|)
Independent arbitrary precision	100-digit mpmath numerical oracle
Maximum absolute deviation	Approximately 2.988 × 10^-15
Batch invariance	Byte-identical result CSVs at batch sizes 1, 37 and 64

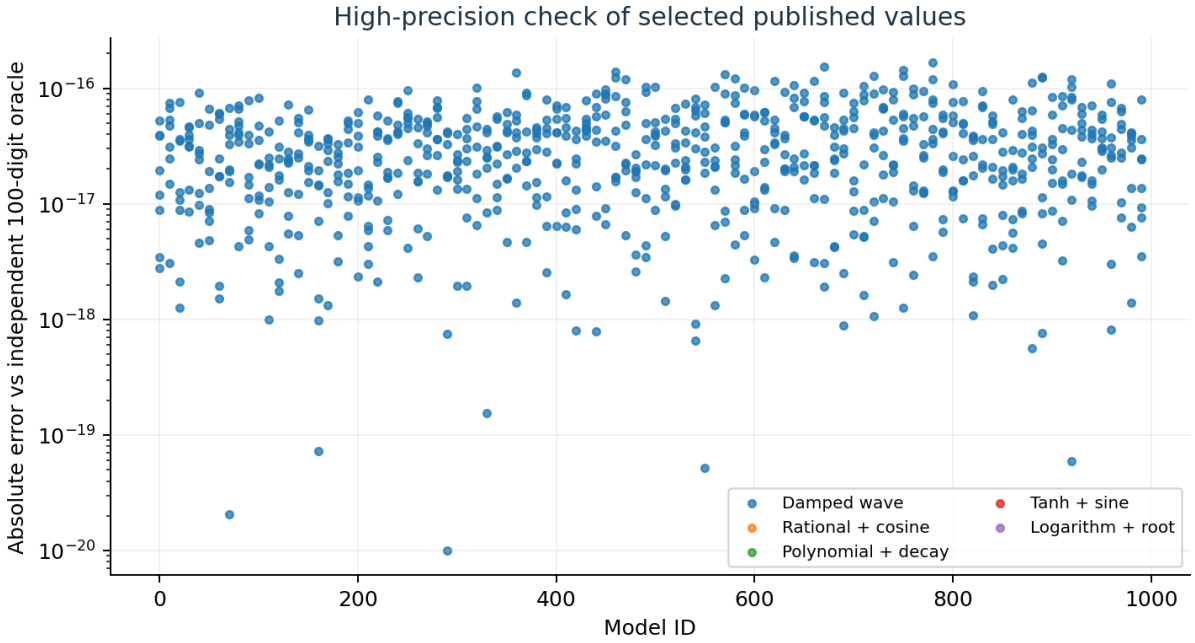


Figure 5. A reproducible subset of absolute deviations against the 100-digit independent oracle.

Small deviations from an exact real-number reference are expected when using double precision. The individual tests do not establish a global numerical error bound or justify any particular scientific model.

5. Versioned mathematical maintenance

Version management is a core part of the on-device use case. To demonstrate a safe update, one pre-approved expression is replaced by a graph whose constant term has changed, without restarting the mathematical engine or rebuilding any other graph. The 16 comparison pairs all reproduce the prescribed 0.125 difference.

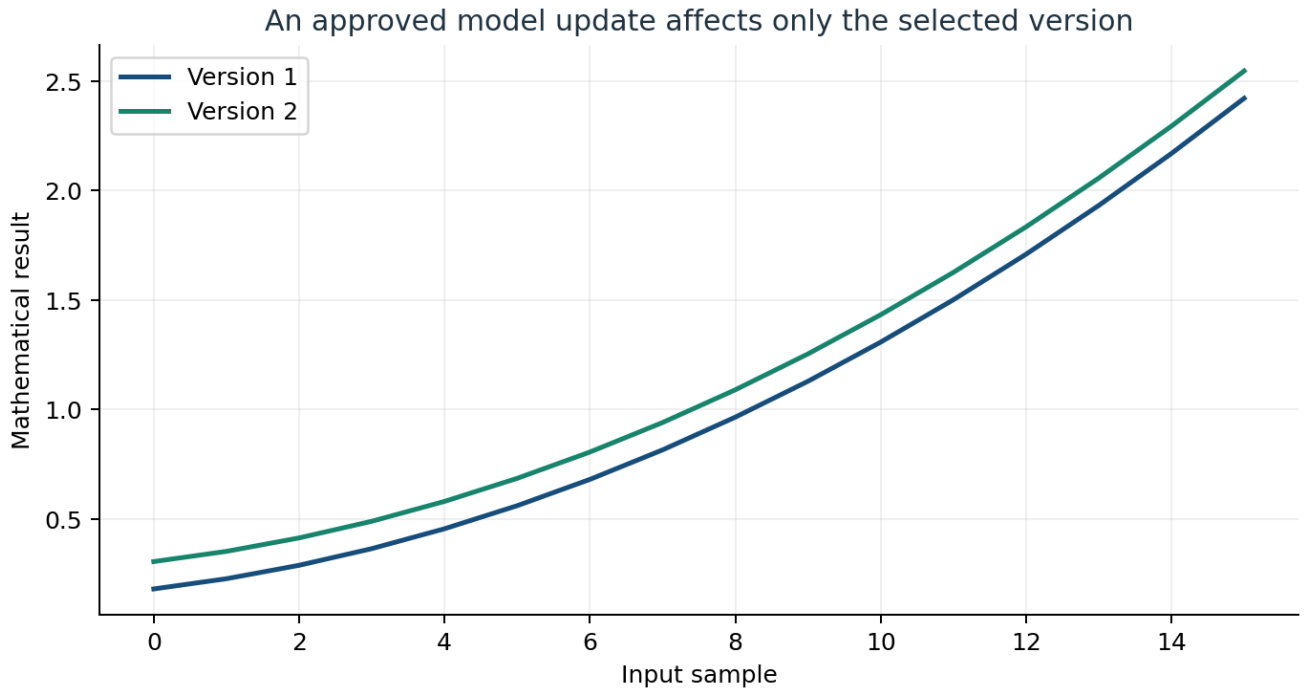


Figure 6. The version-2 computation differs from version 1 by the approved constant shift.

The model-update log includes model identity, input sample, source version, replacement version, old/new result and deviation from the expected change. The other 999 graph handles are retained in memory. This tests formula-scoped change management, not durable persistence across process restarts or multi-tenant configuration governance.

A useful follow-up on a real device would examine restart recovery, memory ceilings for large catalogs, corruption detection and device-level worst-case latency. No such guarantees are claimed here.

6. Pointwise TINC and repeated-input evaluation

A separate computation object enables the optional Trusted Interval Computation mode for the first mathematical family. Its returned intervals are checked against independently computed 100-digit values for 24 sampled inputs. All 24 of these specific references lie inside the corresponding reported binary64 bounds.

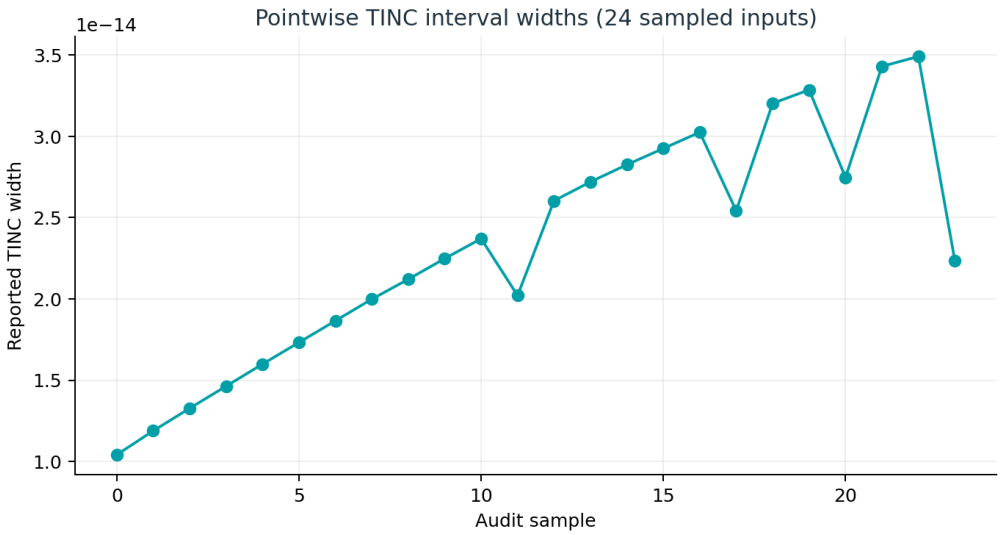


Figure 7. Widths of the 24 actual reported TINC intervals.

This is a pointwise numerical observation for selected samples and does not amount to a certified enclosure theorem for all mathematical functions or hardware platforms.

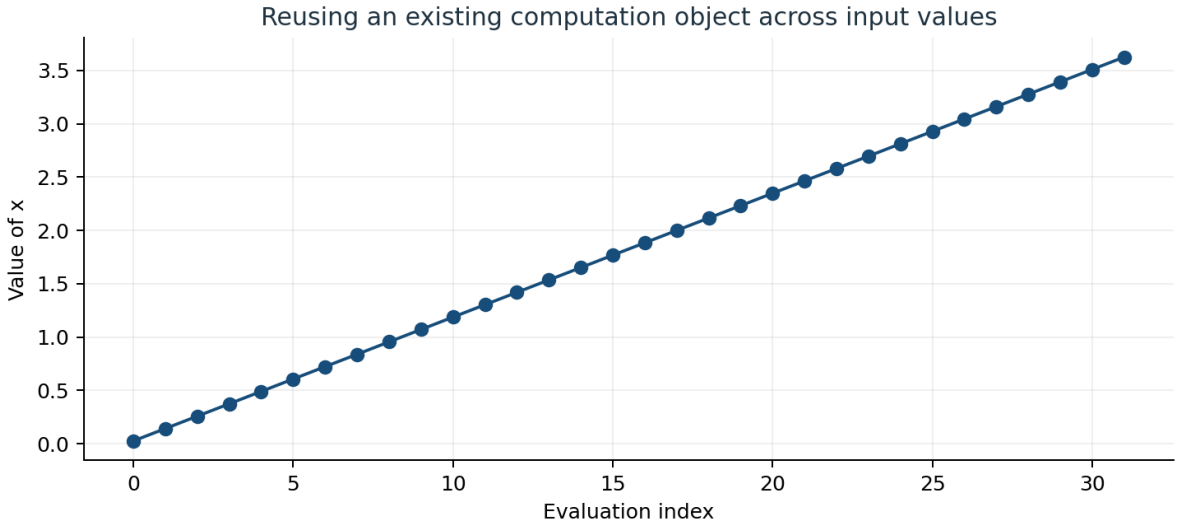


Figure 8. One prepared computation object receives many distinct bounded inputs.

7. Deployment, architecture and reproducibility

Executed environment. The original paceval shared library was supplied externally and linked to an executable compiled with GCC 14.2 on an x86-64 Linux environment. Both GCC and Clang 17 passed three automated CTests; sanitizer-enabled GCC testing also passed all three checks. The complete 1,000-formula scenario was run and its outputs independently audited.

ARM boundary. The official paceval repository and service documentation describe ARM64 deployments. This result does not establish the behavior of the current C++ executable on an ARM device. Deployment requires a compatible ARM64 runtime, ARM64 compiler/ABI configuration and full repetition of the tests on the chosen target. Cross-compiled binaries alone do not demonstrate runtime correctness or operating conditions.

Reproduction. The project contains `src/on_device_runtime.cpp`, `src/paceval_main.h`, `src/paceval_main.cpp`, `src/CMakeLists.txt`, `src/high_precision_reference.py`, example CSV files and `figures/`. Build from `src/` with `-DPACEVAL_RUNTIME` set to the absolute path of a compatible externally acquired binary library. Run the CTests, then execute the default 1,000-model scenario and the Python reference audit as documented in `README.md`.

No measured break-even reuse count, time advantage, p99 latency, RAM ceiling or energy reduction is reported. These quantities were proposed as future system-level evaluation factors in WP-008 and require controlled target-device measurements.

Selected sources

1. paceval API, prepared computations and batch execution: https://paceval.com/api/
2. paceval C++ and architecture references: https://github.com/paceval/paceval
3. ONNX Runtime mobile deployment considerations: https://onnxruntime.ai/docs/tutorials/mobile/
4. Clang cross-compilation and sysroot requirements: https://clang.llvm.org/docs/CrossCompilation.html
5. Raspberry Pi OS 64-bit architecture documentation: https://www.raspberrypi.com/documentation/computers/os.html
6. Source proposal: WP-008, On-Device Reusable Expression Runtime (11 November 2026), user-supplied research proposal.

Scientific conclusion

A reusable, versioned, application-governed mathematical expression runtime is feasible using the original paceval C++ SDK on the tested x86-64 Linux environment. The result provides a reproducible starting point for hardware-specific edge evaluations without implying measurements or qualifications that have not occurred.